Predicting power generation based on regional microclimate data

Shih-Sian Chao
Data Science Degree Program

Yu-Lin Cheng
Department of Economics

Hsuan-Yi Su
Department of Accounting

Abstract

We propose a novel approach to predict solar power generation utilizing microclimate data from observation devices. By implementing comprehensive data processing techniques, including temporal embedding and adaptive normalization, we explore both forecasting and imputation methodologies across various deep learning architectures. Through extensive model evaluation and hyperparameter optimization, TimesNet emerged as the superior model, particularly excelling in imputation tasks. Our comprehensive approach demonstrated competitive performance in a large-scale power prediction competition.

1 Introduction

We participated in a power prediction competition that centers on the challenge of predicting solar panel power generation using microclimate environmental factors. The task involves leveraging meteorological data, recorded at one-minute intervals, to forecast power generation (in watts) every 10 minutes from 9:00 AM to 5:00 PM. Participants are given data from the previous day up to 9:00 AM to make a total of 48 predictions, with the final score based on total error. We are proud to have ranked 44th out of 500 participants.

At the beginning of the competition, we initially used a forecasting methodology to predict the power of the target time periods. Later, we discovered that data existed for the day following nearly every target time period. This observation led us to change our approach to imputation, allowing us to use more information to predict the power of the target time periods.

In this report, we share our approach and findings. We start by highlighting the key articles that inspired our methodology and describing how we processed the data to ensure accuracy and consistency. Our exploration included testing several models and methods, with TimesNet standing out as the most effective. Along the way, we fine-tuned hyperparameters and carefully evaluated performance to refine our predictions. The report concludes with a discussion of our results and the lessons we learned from the competition.

2 Related Work

Temporal pattern modeling plays a crucial role in time series analysis. Earlier works in this domain applied statistical methodologies, such as ARIMA [1] and Kalman Filter [2]. These methods establish foundational frameworks for modeling temporal dependencies through mathematical principles. However, their effectiveness diminishes when encountering the complexity and non-linearity inherent in real-world temporal data. Recent advancements in deep learning have revolutionized time series modeling approaches. Deep learning architectures have significantly improved temporal modeling capabilities. The LSTM architecture [3] revolutionized sequence modeling by introducing memory mechanisms that preserve long-term dependencies. Recently, TiDE [4] demonstrated that well-designed neural architectures could effectively process temporal signals by incorporating domain-specific features. Notable progress has been achieved by TimesNet [5], which introduced an innovative

perspective on temporal modeling through dimensional transformation. The emergence of attention-based models has further advanced the field, with architectures like Crossformer [6] establishing new benchmarks through multi-level temporal relationship modeling, and PatchTST [7] introducing efficient segment-based processing strategies.

Contemporary time series analysis addresses diverse analytical objectives. The fundamental task of forecasting seeks to project future temporal patterns based on historical observations. Imputation focuses on recovering missing temporal values, a critical requirement in real-world applications where data completeness cannot be guaranteed. The field also encompasses specialized tasks such as detecting temporal anomalies and classifying temporal sequences according to their characteristics. Our research specifically addresses forecasting and imputation challenges. While these objectives share underlying requirements in temporal pattern recognition, they present distinct technical challenges: predictive modeling for forecasting versus contextual reconstruction for imputation. Based on extensive analysis of current research [8] [9], we have selected and implemented models that excel in these specific domains, utilizing their advanced capabilities in temporal pattern analysis.

3 Data Processing

In this competition, the dataset was collected using approximately 20 identical observation devices, covering the period from January to November 2024. It was recorded at a per-minute frequency and included parameters such as wind speed (m/s), atmospheric pressure (hPa), temperature (°C), relative humidity (%), illuminance (lux), and power generation (W). To create a complete dataset, we consolidated data from multiple locations, including additional datasets for specific sites. The reason why we merged data is that we thought the patterns were the same in every location. Sunlight intensity (Sunlight(Lux)) observations had an upper limit, so we reconstructed this data. For values below the threshold, regression on power generation data was used to estimate sunlight. For values exceeding the threshold, sunlight was back-calculated using daily power generation values.

To standardize the data, sunlight intensity was scaled using a min-max normalization approach. Since all observation instruments were identical, we assumed that location-specific differences were already reflected in other variables, so the location feature was dropped. Wind speed was excluded as its missing data proportion exceeded 75%. Time information was processed by formatting the `DateTime` column, setting it as the index, and filtering to include data from 7:00 AM to 4:59 PM. The data was split into training (80%), validation (19.9%), and testing (0.1%) sets chronologically. It was then averaged into 10-minute intervals, with periods containing more than three missing values marked as NA. External datasets were merged after averaging to ensure consistency. Solar irradiance values were incorporated by mapping them from external files to corresponding timestamps, enriching the dataset with this important environmental factor. Additionally, we attempted to integrate air pollution data (SO_2, CO, O_3, $PM_{2.5}$) from the Ministry of Environment, but this integration did not improve performance during training and testing.

Before feeding the input data into the model for training, we first perform normalization and then map it to a high-dimensional space through three types of embeddings [8]. The process is defined as follows:

$$H_t = \text{Projection}(X_t) + \text{PE}(X_t) + \text{TE}(X_t^{\text{mark}})$$

where $H_t \in \mathbb{R}^{T \times d_{\text{model}}}$ and d_{model} is the dimension of the embedded representation, value projection $\text{Projection} : \mathbb{R}^C \mapsto \mathbb{R}^{d_{\text{model}}}$ is implemented using a 1D convolution layer, and timestamp embedding $\text{TE} : \mathbb{R}^D \mapsto \mathbb{R}^{d_{\text{model}}}$ is implemented using channel-dimension linear layers. $\text{PE}(\cdot)$ denotes the absolute position embedding. The value projection maps multivariate input values into a high-dimensional space, preserving the multidimensional features of the data and enabling the model to better learn and provide contextual information. The position embedding incorporates absolute positional encoding, which ensures that even after shuffling mini-batches, the model retains the relative positional relationships within the time dimension. The timestamp embedding transforms timestamp information into high-dimensional representations, capturing temporal-specific contextual features and enhancing the model's understanding of temporal data.

4 Models and Methods

Although LSTM has been widely recognized for its utility, as described in the book [10], "The LSTM has been found extremely successful in many applications, such as unconstrained handwriting recognition (Graveset al., 2009), ...", models like TimesNet have emerged as promising alternatives. TimesNet is claimed to achieve state-of-the-art performance across diverse time series tasks by effectively capturing both short-term and long-term dependencies. In this section, we will briefly introduce the structure of TimesNet. We will then explain the forecasting and imputation methodologies employed in this competition.

TimesNet introduces a novel approach to time series analysis by transforming 1D temporal data into 2D tensors, effectively enabling both intra-period and inter-period variation modeling. Its core component, the TimesBlock, first discovers multi-periodicity using Fast Fourier Transform (FFT) to segment time series into periods. These periods are reshaped into 2D tensors where rows and columns represent inter-period and intra-period variations, respectively. TimesBlocks utilize parameter-efficient inception [11] modules with multi-scale 2D kernels to process these tensors, capturing variations across multiple time scales. Post-processing, the tensors are restructured back into 1D representations, facilitating integration with subsequent layers. The multi-period outputs are adaptively aggregated based on their respective frequency amplitudes.

Forecasting. In the first step, the sliding window technique, commonly used in time series tasks, was employed to divide the data into multiple windows. It is important to note that many NAs were present after data preprocessing, and these were skipped during the sliding window process. Each window was further split into two segments based on temporal order. The first segment represented past observations, while the second segment served as the prediction target. The objective was to use the first segment to accurately predict the power of the second segment. Consequently, the first segment was used as the input to the time series model, which then output the predicted power of the second segment. The loss was calculated using the following formula:

$$Loss\left(\hat{x}, x\right) = \left(x - \hat{x}\right)^2,$$

where x denotes the true power of the second segment, and $\hat{x}$ represents the predicted power generated by the time series model. Mini-batch training was applied to enhance computational efficiency and improve model robustness, with the loss averaged over the mini-batch.

Imputation. The first step was identical to the one used in forecasting. Each window was further divided into three segments based on temporal order: the front segment represented past observations, the back segment represented future observations, and the middle segment served as the prediction target. The objective was to use both the front and back segments to predict the power of the middle segment. To achieve this, the values of the middle segment were replaced with zeroes, as only the front and back segments were used for input. The modified middle segment was then included as part of the input to the time series model, which subsequently output the predicted power of the middle segment. The loss was calculated using the formula:

$$Loss\left(\hat{z}, z\right) = \left(z - \hat{z}\right)^2,$$

where z denotes the true power of the middle segment, and $\hat{z}$ represents the predicted power generated by the time series model. Mini-batch training was also applied for imputation.

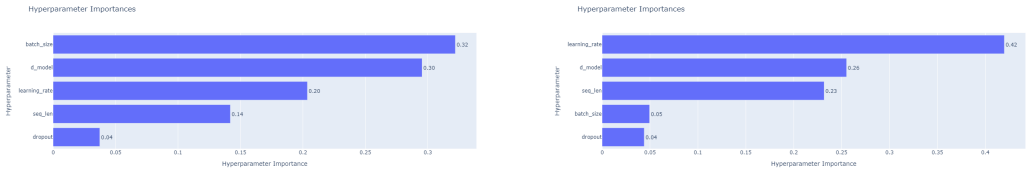
5 Hyperparameter Tuning

Table 1: Hyperparameter ranges for imputation and forecasting

Hyperparameter	Imputation range	Forecasting range
Sequence length	120–180, step 2	40–72, step 2
Dimension model†	16–64, step 2	
Batch size	8, 16, 32, 64	
Learning rate	1e-4–1e-3*	
Dropout	0.1–0.3, step 0.05	

† Used in the embedding layer to map time series data into a higher dimensional space.
* Sampled from a log space to ensure all values have an equal chance of being selected.

For hyperparameter optimization, as detailed in Table 1, we employed the Tree-structured Parzen Estimators (TPE) algorithm, a sequential model-based optimization technique. Each model was evaluated across 10 trials with hyperparameter combinations suggested by the TPE optimizer, running for 50 epochs per trial to ensure robust evaluation and fair comparison. The evaluation metric used for hyperparameter tuning was the validation loss, measured by the Mean Absolute Error (MAE).



(a) Forecasting

(b) Imputation

Figure 1: Hyperparameter importance analysis for (a) forecasting and (b) imputation tasks.

To better understand the impact of individual hyperparameters, we conducted a hyperparameter importance analysis to evaluate their impact on forecasting and imputation tasks. As shown in Figure 1, it reveals distinct sensitivity patterns between forecasting and imputation tasks. For forecasting, batch size demonstrated the highest importance (0.32), followed by embedding dimension and learning rate. In contrast, learning rate emerged as the most critical factor for imputation (0.42), with embedding dimension and sequence length also playing significant roles. Notably, embedding dimension was consistently essential across both tasks, while dropout rate showed minimal influence. These results underscore the necessity of task-specific tuning strategies.

6 Results

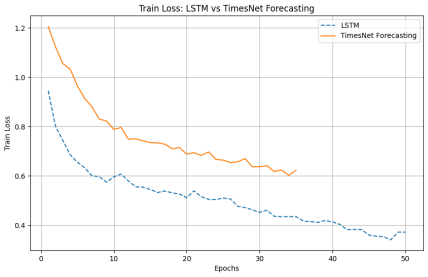
In this section, we present the results of various time series models utilized in this competition, focusing on their performance with forecasting and imputation techniques. The evaluation includes (1) a table summarizing the training and validation losses of multiple models for both methodologies, (2) comparative figures illustrating the training and validation losses of LSTM versus TimesNet, and (3) visualizations of the prediction results using TimesNet for both forecasting and imputation. To emphasize the best-performing models, we highlight the lowest loss values in red in Table 2. It is important to note that LSTM serves as our baseline model, but it was not employed for the imputation technique. Additionally, the training and validation loss values depicted in the figures of imputation represent the outcomes of forecasting.

Table 2: Performance metrics for various models.

Model	**Forecasting**		**Imputation**	
	Training Loss	**Validation Loss**	**Training Loss**	**Validation Loss**
LSTM	**0.3833**	0.5167	-	-
Crossformer	0.7457	0.7197	0.3909	0.2690
Nonstationary Transformer	0.8235	0.7689	0.6990	0.5948
TimesNet	0.6232	0.5317	**0.3773**	**0.2281**
PatchTST	0.7562	**0.4046**	0.8367	0.3253
TiDE	0.8208	0.4987	0.6085	0.4478

* The red color is used to denote the best results.

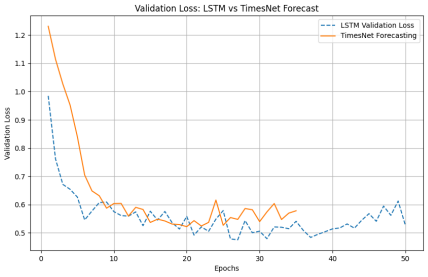
From Table 2, it is evident that the performance of imputation consistently surpasses forecasting in terms of validation loss. This can be attributed to the fact that imputation uses not only past



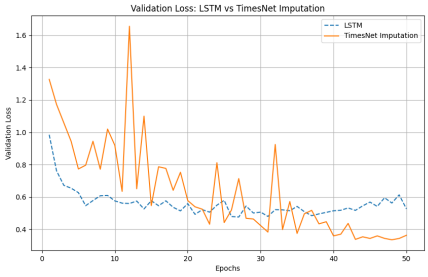
(a) Forecasting

(b) Imputation

Figure 2: Comparison of the train loss of LSTM versus TimesNet



(a) Forecasting



(b) Imputation

Figure 3: Comparison of the validation loss of LSTM versus TimesNet

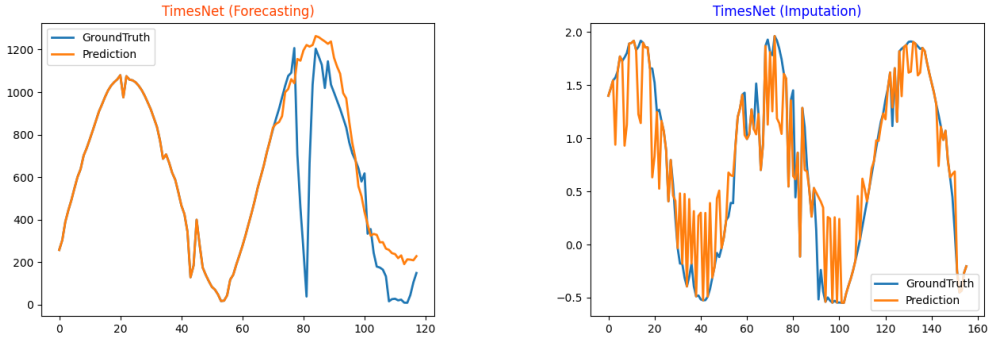
observations but also future observations, which provides additional information and allows it to achieve lower validation losses. This advantage is further supported by the comparative figures in Figure 4. As a widely-used time series model, LSTM demonstrates its powerful capability in forecasting, evident from its low training and validation losses shown in Table 2. The comparative figures also reveal LSTM's stable training process and its ability to achieve low training loss from the very beginning, reinforcing its reliability in forecasting tasks.

On the other hand, TimesNet stands out in imputation, achieving the best training and validation losses (0.3773 and 0.2281, respectively). These results align with the conclusions of the review paper [8], which highlights TimesNet's superior performance in imputation. However, the results of the Nonstationary Transformer contradict the claims made in the review paper. While the review paper claims it ranks among the top two models for imputation, our results indicate that it performed the worst in this task. This inconsistency may arise from differences in datasets and task setups. For different datasets and tasks, it is crucial to select the most suitable model.

7 Conclusion and Future Work

In this study, we elaborate on our approach to the power prediction competition. We analyzed regional microclimate data and performed feature engineering. Then, we studied recent review papers on time series, from which we learned about widely used models such as LSTM and advanced models like TimesNet. In addition to these models, we learned methodologies for forecasting and imputation to address time series prediction problems. Based on this knowledge, we made predictions on power generation with careful hyperparameter tuning. Finally, we presented and analyzed the results.

We also identified several deficits in this work. First, due to limited time, we restricted training epochs to fewer than 150 in this competition, which may have limited performance. Higher training epochs could potentially yield better results. Second, as described in [5], replacing the inception blocks with more powerful CNN blocks, such as those in ResNeXt [12], could enhance performance. Third, advanced time series models based on TimesNet could further improve results. For exam-



(a) Forecasting

(b) Imputation

Figure 4: Visualization of the prediction results of forecasting versus imputation using TimesNet

ple, [13] proposed a model that integrates three time-series structures—TimesNet, Crossformer, and LSTM—demonstrating the potential to surpass TimesNet's performance through experiments. Overall, with these findings, we expect to achieve better results in future time-series tasks.

References

[1] Oliver D. Anderson and M. G. Kendall. Time-series. 2nd edn. *The Statistician*, 25:308, 1976.

[2] Rudolph Emil Kalman. A new approach to linear filtering and prediction problems. *Transactions of the ASME–Journal of Basic Engineering*, 82(Series D):35–45, 1960.

[3] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Comput.*, 9(8):1735–1780, November 1997.

[4] Abhimanyu Das, Weihao Kong, Andrew Leach, Shaan Mathur, Rajat Sen, and Rose Yu. Long-term forecasting with tide: Time-series dense encoder, 2024.

[5] Haixu Wu, Tengge Hu, Yong Liu, Hang Zhou, Jianmin Wang, and Mingsheng Long. Timesnet: Temporal 2d-variation modeling for general time series analysis. *arXiv preprint arXiv:2210.02186*, 2022.

[6] Yunhao Zhang and Junchi Yan. Crossformer: Transformer utilizing cross-dimension dependency for multivariate time series forecasting. In *The Eleventh International Conference on Learning Representations*, 2023.

[7] Yuqi Nie, Nam H. Nguyen, Phanwadee Sinthong, and Jayant Kalagnanam. A time series is worth 64 words: Long-term forecasting with transformers, 2023.

[8] Yuxuan Wang, Haixu Wu, Jiaxiang Dong, Yong Liu, Mingsheng Long, and Jianmin Wang. Deep time series models: A comprehensive survey and benchmark. *arXiv preprint arXiv:2407.13278*, 2024.

[9] Jun Wang, Wenjie Du, Wei Cao, Keli Zhang, Wenjia Wang, Yuxuan Liang, and Qingsong Wen. Deep learning for multivariate time series imputation: A survey. *arXiv preprint arXiv:2402.04059*, 2024.

[10] Ian Goodfellow. Deep learning, 2016.

[11] Christian Szegedy, Wei Liu, Yangqing Jia, Pierre Sermanet, Scott Reed, Dragomir Anguelov, Dumitru Erhan, Vincent Vanhoucke, and Andrew Rabinovich. Going deeper with convolutions. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 1–9, 2015.

[12] Saining Xie, Ross Girshick, Piotr Dollár, Zhuowen Tu, and Kaiming He. Aggregated residual transformations for deep neural networks. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 1492–1500, 2017.

[13] Jun He, Kuidong Yuan, Zijie Zhong, and Yifan Sun. Based on timesnet-crossformer-lstm short term power load forecast. *IEEE Access*, 2024.